Dissertation for the Degree of Doctor

A Study of Resource Structure for Distributed AI in Edge Computing

by
Kwon Yong-Hun

Department of Computer Information
& Telecommunications Engineering

Graduate School
Kangwon National University

February, 2026

Supervised by
Professor Jung In-Bum

A Study of Resource Structure for Distributed AI in Edge Computing

A DISSERTATION
Submitted to the Graduate School of
Kangwon National University in Partial
Fulfillment of the Requirements
for the Degree of

Doctor of Engineering

by
Kwon Yong-Hun

Department of Computer Information
& Telecommunications Engineering

February, 2026

Approved by Committee of the Graduate School of Kangwon National University in Partial Fulfillment of the Requirements for the Degree of Doctor of Engineering

Kwon Yong-Hun

December, 2025

Dissertation Committee ;

Lee Heon-Guil *(signature)*
(Chairman of Committee)

Kim Yong-Seok *(signature)*
(Committeeman)

Song Won-Jun *(signature)*
(Committeeman)

Kim Man-Bae *(signature)*
(Committeeman)

Jung In-Bum *(signature)*
(Committeeman)

A Study of Resource Structure for Distributed AI in Edge Computing

Kwon Yong-Hun

Department of Computer Information & Telecommunications Engineering
Graduate School, Kangwon National University

Abstract

The advancement of cloud computing has significantly contributed to enabling artificial intelligence (AI) services. However, as the demand for AI services continues to grow, the issues of energy consumption and latency in centralized cloud servers have become more pronounced. To address these challenges, edge computing has been proposed. By processing data at the network edge, edge computing enables faster responses and offers security benefits, as data can be handled locally. Nevertheless, the resource constraints and heterogeneity of edge devices present significant challenges in providing diverse AI services.

To overcome these limitations, this paper proposes a resource structure suitable for edge computing environments. A Resource-Cell is a logical and quantitative unit that represents the resources required for a device to execute AI workloads. Unlike theoretical performance metrics, Resource-Cells are derived from actual device performance through a Multi-Dimensional Benchmark. The benchmark consists of four models built from representative operators used in AI workloads, and each benchmark contains ten stages. The computational load and memory access increase at each stage, allowing the benchmark to evaluate models of varying sizes.

To validate the proposed approach, the Raspberry Pi series is used as the

experimental platform. Through experiments, we analyze the accuracy of Resource-Cell estimation, the performance of distributed processing, and the battery efficiency associated with Resource-Cell utilization. The results confirm that the Resource-Cell structure is effective for operating distributed AI workloads in edge environments.

□ keywords

Edge Computing, Distributed AI, Internet of Things, Benchmark, Resource Constraint, Heterogeneous

CONTENTS

Ⅰ. Introduction ····· 1
Ⅱ. Related Work ····· 3
1. Edge Artificial Intelligence ····· 3
2. AI Workload-based Benchmark ····· 5
Ⅲ. Resource-Cell Structure ····· 7
1. Motivation ····· 7
2. Concept of the Resource-Cell Structure ····· 9
3. Resource-Cell Unit ····· 11
4. Device Resource Table ····· 12
5. Summary ····· 14
Ⅳ. Multi-Dimensional Benchmark Suite ····· 15
1. Taxonomy of AI Model Operator ····· 15
2. Benchmark Suite ····· 16
1) GEMM(General Matrix Multiplication) ····· 17
2) Convolution ····· 17
(1) Standard Convolution ····· 17
(2) Depthwise Convolution ····· 18
3) Attention ····· 18
3. Implementation ····· 19
1) Device Setup ····· 19
2) Benchmark Suite Parameter ····· 21
4. Result ····· 22
5. Summary ····· 24
Ⅴ. Evaluation ····· 25
1. Performance Prediction Accuracy ····· 25
1) MobileNet ····· 26
2) EffecientNet ····· 26

3) Vision Transformer ····· 26
4) ResNet ····· 26
2. Power Consumption ····· 27
3. Task Distribution Evaluation ····· 30
1) Load Balance Evaluation at Distributed Processing ····· 32
2) Turnaround Time Evaluation ····· 34
4. Summary ····· 35
Ⅵ. Conclusion ····· 36

LIST OF TABLES

Table 1. Edge Device Resource Information Table ····· 12
Table 2. AI Operator and Representative Model ····· 15
Table 3. Raspberry Pi Series specification ····· 20
Table 4. Benchmark Model Parameter ····· 21
Table 5. Flops and Memory Usage per Benchmark Step ····· 21
Table 6. Resource-Cell Values Computed for Each Operator ····· 23

LIST OF FIGURES

Figure 1. Comparison between Cloud Computing and Edge Computing ····· 1
Figure 2. Conceptual Representation of the Roofline Model ····· 5
Figure 3. Priority Management of Heterogeneous Edge Devices ····· 7
Figure 4. Resource Management using Resource-Cell Structure ····· 9
Figure 5. Structure of Resource-Cell Units in Edge Devices ····· 11
Figure 6. Device Resource Table with Resource-Cell ····· 12
Figure 7. Overview of the Multi-Dimensional Benchmark Suite Pipeline ····· 16
Figure 8. Raspberry Pi with Current Sensor ····· 19
Figure 9. Results of the Benchmark Suite on Each Device ····· 22
Figure 10. Inference Time Prediction in AI Models ····· 25
Figure 11. Power Consumption Model and Performance Evaluation ····· 28
Figure 12. MQTT-based Distributed Processing Architecture with Resource-Cell ···· 30
Figure 13. Execution Screen of Resource Monitoring on the Server ····· 31
Figure 14. Inference Counts per Device Across Task Distribution Strategies ····· 32
Figure 15. Comparison of Turnaround Time Across Task Distribution Strategies ···· 34

Ⅰ. Introduction

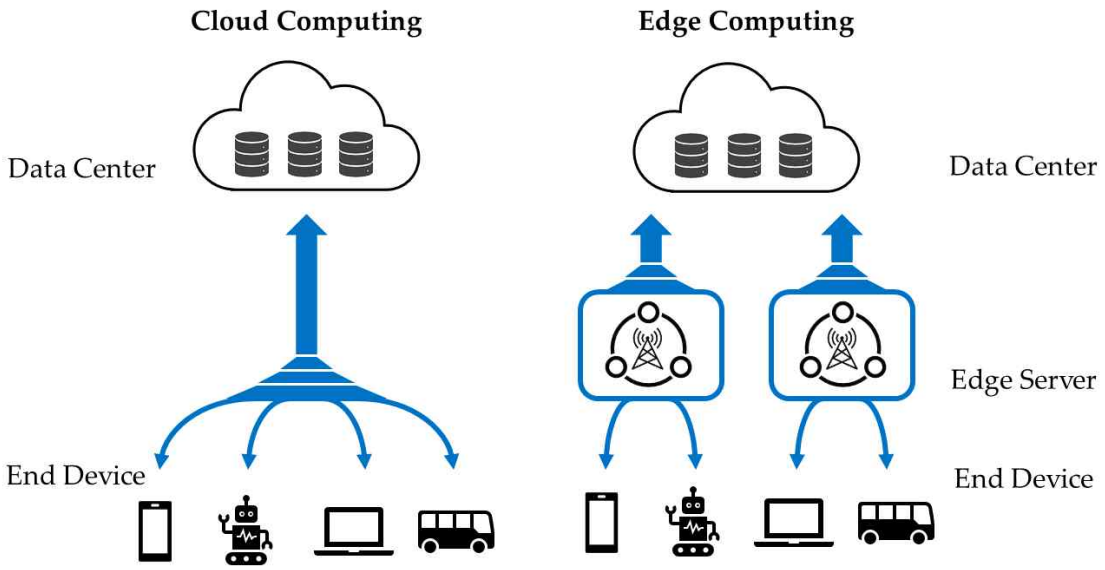

Figure 1. Comparison between Cloud Computing and Edge Computing

The advancement of hardware accelerators and AI models has led to the emergence of numerous commercially successful applications, moving beyond theoretical forms of artificial intelligence. Generative AI models such as ChatGPT are massive in scale, with billions of parameters, and are capable of generating sophisticated images and sentences through extensive data training. Since it is difficult to perform inference tasks for such services on user devices, they are executed in cloud data centers. These systems have developed into architectures that leverage the data center's virtually unlimited computing resources to process user requests and deliver results. Recently, the exponential rise in demand for AI services has accelerated the ongoing expansion and deployment of new data centers. This trend has accelerated the emergence of large-scale data centers powered by AI chips developed by major technology companies, including NVIDIA and Google. In particular, the explosive growth of generative AI services has driven tremendous energy consumption. Supporting such workloads requires extensive investment in power-generation capacity and advanced cooling infrastructure[1,2].

Furthermore, the concentration of a massive number of user requests on cloud servers results in server overload and network bottlenecks, causing latency that ultimately degrades AI service quality. With a centralized architecture, if a data center suffers from a fire or power outage, core systems may be paralyzed, leading to service failure. Additionally, the increasingly frequent issue of personal information leaks remains a potential and persistent problem. To address the limitations of cloud computing, edge

computing has been proposed. Figure 1 illustrates the structure of edge computing. By processing data on devices positioned near the user rather than transmitting it to a centralized server, edge computing significantly reduces latency and enables rapid, real-time responses. It utilizes a hierarchical architecture that processes data before forwarding it to the data center, thereby mitigating the latency and energy overheads inherent in cloud computing[3,4].

However, the resource constaints of edge devices, relative to that of data center servers, makes it difficult to achieve service quality comparable to that of cloud systems. To enable AI services in edge environments, ongoing advancements in both hardware and software are essential. These include the development of chips with resource-efficient architectures, model compression, and distributed processing strategies aimed at maximizing resource efficiency. With continuous advancements in hardware and AI models, the performance gap across devices widens, thereby amplifying the challenge of heterogeneity in edge environments. To handle large-scale tasks in edge environments, it is necessary to form clusters for distributed processing. However, variations in computational power and battery status across devices make heterogeneity a critical constraint in cluster configuration, substantially influencing overall service quality. For example, even if a particular device has high throughput, a lack of network bandwidth or battery capacity can affect both real-time responsiveness and service continuity. Conversely, even when external resources are sufficient, if many devices have low processing capabilities, the overall service response time may be delayed. These cases show that, in edge computing, not only can inefficient resource utilization occur, but also service failures such as interruptions may arise [5 - 7].

This paper proposes a method to mitigate heterogeneity in edge environments and utilize resources more effectively within distributed processing systems. The Resource-Cell is a quantified unit that represents a device's resources, calculated through AI workload - based benchmarking. Rather than relying on theoretical performance, Resource-Cells are derived from multifaceted measurements of actual performance observed during AI inference tasks. By expressing diverse performance characteristics as a single metric, the Resource-Cell enables intuitive prediction and comparison of device capabilities. In distributed processing tasks, accurate performance prediction leads to more effective load balancing, thereby improving overall processing efficiency.

The structure of this paper is as follows. Chapter 2 introduces technologies related to artificial intelligence and benchmarking in edge computing environments. Chapter 3 explains the necessity and concept of the Resource-Cell structure. Chapter 4 presents an AI-based benchmark for calculating resource cells. Chapter 5 discusses the calculation accuracy of Resource-Cells and performance evaluation in distributed processing environments. Finally, Chapter 6 provide the conclusion.

Ⅱ. Related Work

1. Edge Artificial Intelligence

Edge computing offers advantages such as low latency and data security by processing data in real time at edge nodes located near user devices or data-generating devices. In general, edge environments are resource-constrained in terms of computing power, memory capacity, and power supply. Moreover, the edge environment consists of heterogeneous systems due to the coexistence of processors with various architectures and diverse networks. Because of these limitations, resource management and optimization are required when deploying AI models in edge environments [8].

To operate artificial intelligence in edge environments, several approaches have been explored. First, research efforts have focused on designing lightweight AI models that reduce computational load and are optimized for execution on resource-constrained devices. Additionally, various studies have investigated methods to enhance data security by collecting and processing data locally. Second, techniques have been developed to offload computationally intensive tasks to edge clusters or nearby edge servers for distributed processing. Lastly, cooperative learning approaches have been proposed in which neighboring edge devices collaborate with one another [9].

Methods for compressing AI models typically include pruning, which removes redundant nodes, and quantization, which reduces model size by lowering the numerical precision of parameters. Other approaches, such as tensor decomposition and knowledge distillation, are also used. However, these methods often suffer from the drawback of degrading model accuracy[10]. Neural Architecture Search (NAS) is a technique that defines a search space and search strategy, then repeatedly explores and validates candidate models to determine an optimal architecture[11]. Nevertheless, traditional NAS does not account for hardware performance. To overcome this limitation, Hardware-aware NAS (HW-NAS) has been introduced, which searches for architectures based on hardware performance metrics[12]. Despite these improvements, NAS-based approaches generally require substantial computational resources because they involve searching and training numerous candidate models. As a result, they are difficult to deploy in edge environments where available resources are constrained[13].

In edge environments, one method of utilizing artificial intelligence is through distributed processing. The core of distributed processing lies in maximizing resource efficiency. In environments composed of numerous heterogeneous devices, it is crucial to adopt task distribution strategies that comprehensively consider communication costs, computational performance, and memory status of each device [14].

In edge environments, distributed processing is a widely used approach for utilizing artificial intelligence. The core principle of distributed processing is to maximize resource efficiency. In environments composed of many heterogeneous devices, it is essential to employ task distribution strategies that comprehensively consider communication costs, computational performance, and memory availability of each device[14]. For training AI models, the model or dataset can be partitioned and trained in parallel, after which gradients are aggregated on a server to update parameters. In the case of federated learning, training is conducted exclusively with the data generated on each device to ensure privacy[15].

Such methods operate through cooperation among devices or by offloading tasks to edge servers. For example, Neurosurgeon optimized offloading by partitioning the CNN-based model AlexNet at the granularity of layers, thereby minimizing communication overhead[16]. Another study dynamically determined optimal partition points based on computational burden, energy consumption, and output size of each layer in heterogeneous environments[17]. DeepThings proposed a technique that reduces redundant computations by dividing CNN feature maps into tiles and distributing them in a pipeline structure for processing[18]. A recent survey analyzed integrated approaches that combine content caching, offloading, and resource management at edge servers, and optimize them using Deep Reinforcement Learning (DRL)[19].

2. AI Workload-based Benchmark

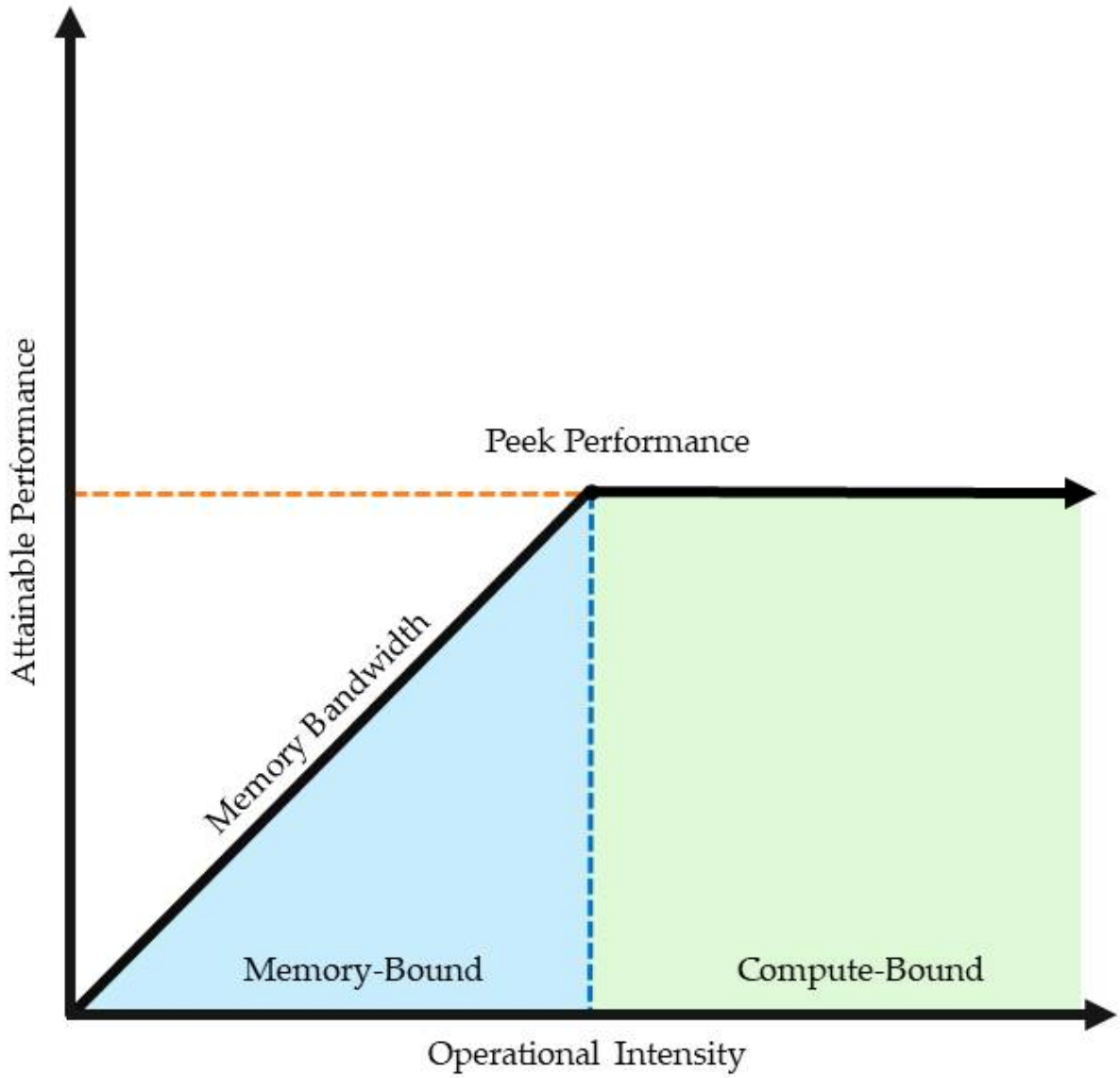

Figure 2. Conceptual Representation of the Roofline Model

When deploying artificial intelligence models in heterogeneous environments, performance bottlenecks occur. Device architectures are designed to perform specific tasks, and bottlenecks may arise for various reasons depending on the model's characteristics. To deploy and execute AI models efficiently in distributed systems, benchmarks capable of quantitatively analyzing device performance characteristics and resource bottlenecks are required[20]. A benchmark refers to a tool that quantitatively evaluates and compares the performance of hardware or software systems using a specific methodology. Since benchmarks serve as important references for many users and researchers, reliability and reproducibility are essential requirements for any

benchmarking process [21].

MLPerf Inference, a representative AI benchmarking tool, is designed to evaluate a device's computational performance through various workloads such as image classification and object detection. AI Benchmark and TFLite Micro Benchmark are lightweight tools used in edge environments that can measure memory usage and power efficiency [22-24].

In particular, in edge environments where heterogeneous devices coexist and resources are limited, benchmarking tools must extend beyond a single metric and incorporate composite, workload-based evaluations that account for both compute-intensive and memory-intensive characteristics. The Roofline model, which theoretically analyzes device performance, combines the operational intensity of an application with the device's memory bandwidth and computational performance limits. This helps visually determine whether a model is compute-bound or memory-bound[25]. However, since factors such as power consumption, cache memory size, and other parameters also influence bottlenecks beyond simple memory access and computational load, more precise models are needed to calculate the causes and extent of the bottlenecks.

Benchmark results are often difficult for end users to reproduce, and because measurement criteria vary by benchmarking tool, the consistency of results can be limited [26]. Therefore, in heterogeneous environments such as the edge, there is a need for benchmarking models that are simple, intuitive, and capable of being executed quickly.

Ⅲ. Resource-Cell Structure

1. Motivation

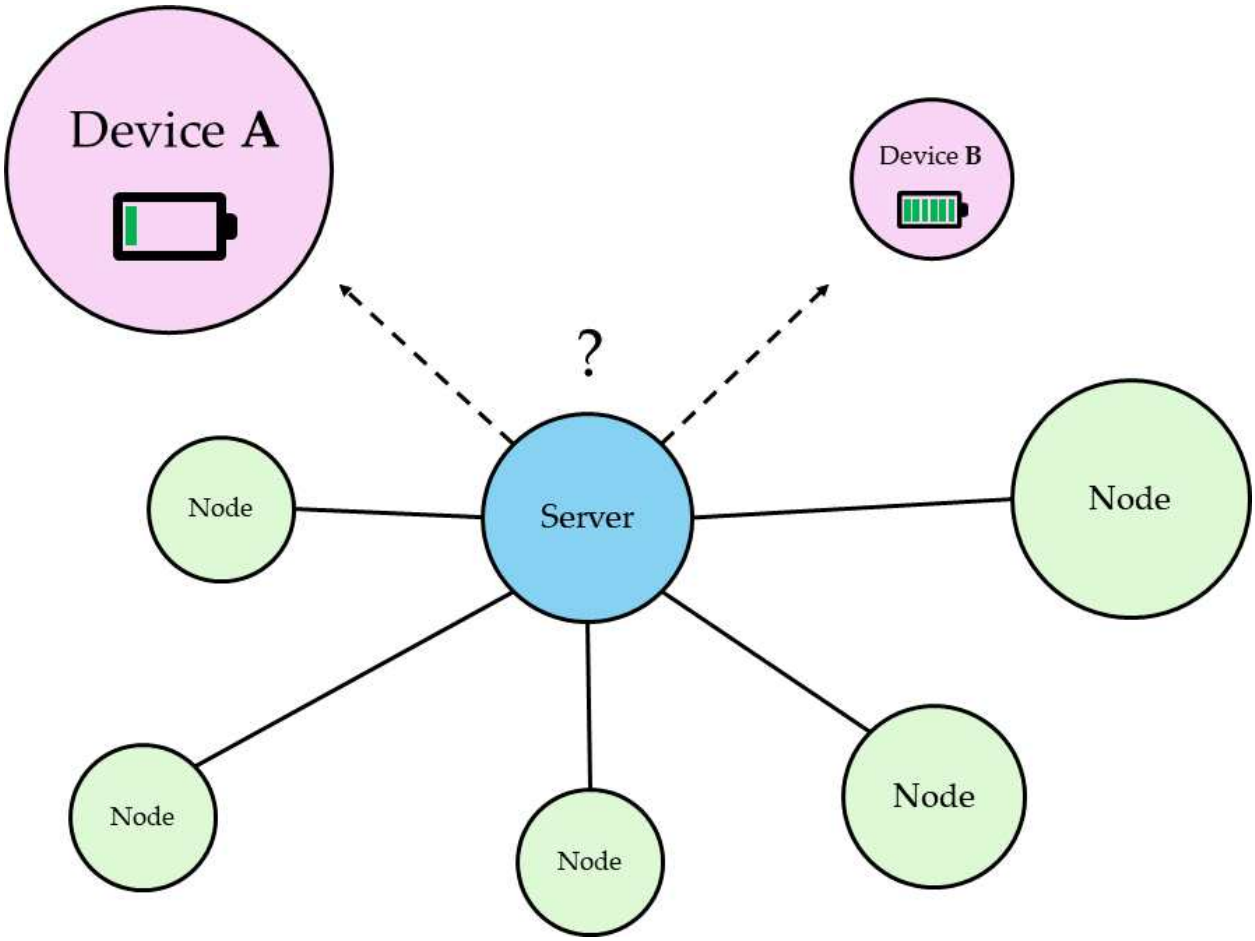

Figure 3. Priority Management of Heterogeneous Edge Devices

Edge computing is a paradigm proposed to address issues such as latency and single points of failure (SPOF) in cloud computing. It enables fast and secure data processing by utilizing resources located near data sources or users. However, unlike large-scale data centers in the cloud, edge environments are resource-constrained. Moreover, the coexistence of various processor architectures and diverse network types creates a heterogeneous environment.

Therefore, to reliably provide AI services, it is essential to comprehensively consider various resource factors such as the computational performance of edge devices, network bandwidth, and battery status. A deficiency in any one of these resources can undermine system stability and efficiency, ultimately degrading the overall quality of service (QoS).

For example, as illustrated in Figure 3, if the computational performance of a device is visualized as the size of a circle, Device A may have high computational performance but insufficient remaining battery power, potentially leading to service interruption due to power depletion during task execution. This becomes a factor that degrades service quality in terms of longevity. In contrast, Device B may have sufficient battery capacity but low computational performance, making it unable to complete assigned tasks within the required timeframe. This could lead to processing delays across the entire edge network.

As such, there is a need for research that considers the resource-constraints of edge computing, ensures the longevity of AI services, and effectively utilizes the remaining resources of devices deployed in edge regions.

This study proposes a method for quantifying and expressing the resources of edge devices. A Resource-Cell is calculated based on the computational performance of processors and memory capabilities embedded in the device. These performance metrics are measured through benchmarking based on AI workloads.

Unlike theoretical performance, the actual performance of a device may be constrained due to various bottlenecks, and achievable performance often falls short. Performance also varies depending on the structure of the AI model; some models may experience compute-bound bottlenecks, while others may be memory-bound. To reflect these requirements, multi-dimensional benchmarking is performed.

Based on the results, Resource-Cells are calculated, and each cell exhibits different performance characteristics depending on the type of computation.

In this study, Resource-Cells are used to predict the inference time of AI models. Task scheduling is then performed based on the predicted performance, and improvements are evaluated by comparing it with traditional scheduling methods. In addition, power measurement modules embedded in the devices are used to verify the power efficiency of the Resource-Cell structure.

2. Concept of the Resource-Cell Structure

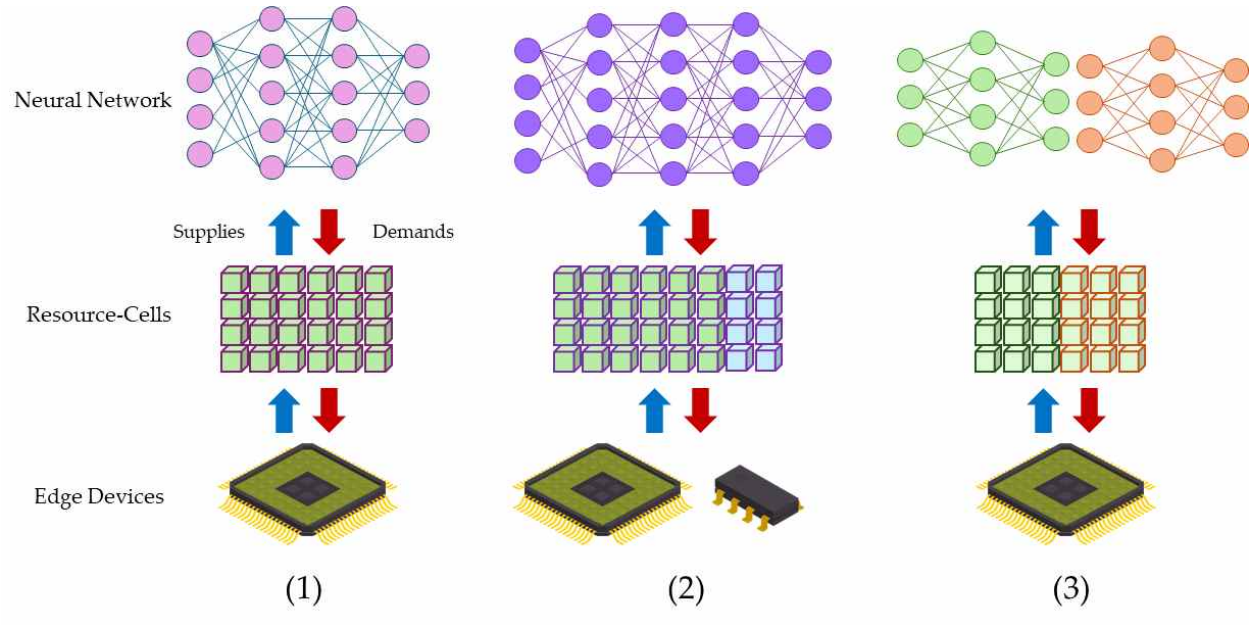

Figure 4. Resource Management using Resource-Cell Structure

Figure 4 illustrates how AI service models are operated using the Resource-Cell concept. The resources available on a device are represented as multiple cells, as shown in the figure, and each cell reflects the absolute performance capacity of the device. AI models, in order to guarantee real-time performance, also require a certain number of Resource-Cells. In this figure, both the resources required by the AI model and the resources possessed by the edge device are quantified using the same unit—Resource-Cells.

Figure 4 (1) represents a case where the Resource-Cells available on a device exactly match or exceed the number required by the AI model. This indicates that the model can be operated independently on a single device, making on-device deployment possible without the need for distributed processing.

Figure 4 (2) illustrates a scenario in which the model's resource demand exceeds the capacity of a single device. In this case, multiple devices share resources and perform distributed processing. The system searches for other devices with the required number of Resource-Cells and distributes tasks proportionally according to the number of cells each device has.

Figure 4 (3) shows a case where the device has more Resource-Cells than the model requires, demonstrating the possibility of running multiple models in parallel on a single device.

The Resource-Cell structure enables a direct comparison between the available resources of a device and the resource requirements of a model using a common unit. This allows for predicting and comparing the execution time of tasks on each device, thereby achieving load balancing during task scheduling. Furthermore, this visualization helps users intuitively understand the performance of edge devices, enabling them to configure environments appropriately based on their specific needs.

3. Resource-Cell Unit

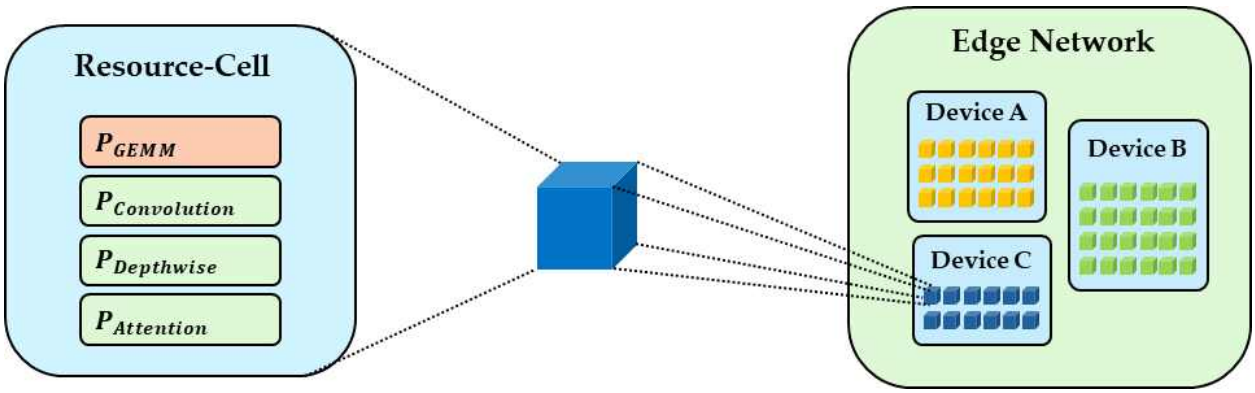

Figure 5. Structure of Resource-Cell Units in Edge Devices

A Resource-Cell is a quantified unit that represents a device's resources. Its fundamental component is computational performance, measured based on CPU capability and memory bandwidth. Resource-Cells are calculated using the actual computational load that an edge device can handle.

However, in AI models, there are operations with different structures and memory access patterns, such as convolutional neural networks, beyond simple matrix multiplication. While a device's performance can be mathematically calculated using theoretical values, various factors such as differences in compute units depending on CPU architecture, cache size, and power status can cause computational bottlenecks. Consequently, even when two models have the same computational load, their performance may vary depending on structural characteristics, even under identical conditions. Furthermore, performance differences can arise within the same model depending on whether quantization is applied. A Resource-Cell addresses this variability by employing the multi-benchmark procedure that reflects diverse model structures, rather than relying on a single benchmark.

Figure 5 illustrates the structure of the Resource-Cell unit. Edge devices within an edge network contain multiple Resource-Cells, each representing performance for specific types of operations. Benchmarking varies both the computational load and memory access patterns to measure realistic inference performance. The benchmark suite consists four types of benchmarks and is normalized based on the results of the GEMM benchmark model. In this paper, this benchmark is referred to as the Multi-Dimensional Benchmark Suite, and the GEMM benchmark serves as the Reference Benchmark in this suite. The detailed configuration of the benchmark is described in Section Ⅳ.

4. Device Resource Table

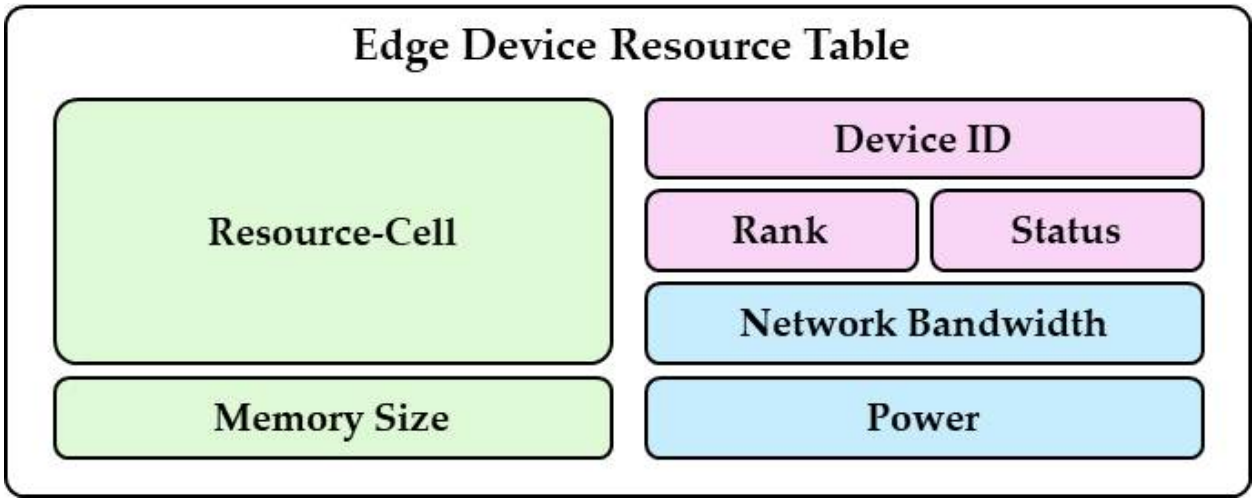

Figure 6. Device Resource Table with Resource-Cell

Table 1. Edge Device Resource Information Table

Category		Description
Device	Device ID	n
	Status	Ready, Waiting, Running
	Task	Inference, Benchmark, etc.
	Rank	Server, Client0, Client1,⋯
Resource	Resouce-Cell	$[P_{GEMM}, P_{Conv}, P_{Depth}, P_{Attn}]$
	Memory Size	MB
Environment	Network Bandwidth	bps/s
	Power	mAh

Figure 6 shows the Resource table, which represents the resources available on edge devices. It is used to access the status of each device and serves as foundational data for distributed processing. Table 1 summarizes these elements according to their respective categories.

Device primarily consists of the device's ID and status. The Resource-Cell structure is a concept designed for resource-based task scheduling in distributed processing environments. Therefore, each device is assigned a unique identifier to facilitate seamless distributed processing. Status field represents the current operational state of the device and is categorized into Ready, Waiting, and Running. Ready indicates that the device is idle and capable of accepting tasks. Waiting indicates that a task has already been assigned and the device is awaiting the next input. Running signifies that the device is currently processing input data. The task field represents the command or operation the device is currently executing. Rank indicates the device's position within the network.

Resource describes the available capabilities of a device, including its Resource-Cells, which represent computational capacity as determined by through benchmark results. Each cell value is normalization based on GEMM benchmark and varies depending on the operator type. Memory Size is used to determine whether the device can accommodate a specific AI model. Even if sufficient Resource-Cells are available to perform inference, the memory size must still be checked to ensure that the model can actually be executed on the device.

Environment variables represent the factors that influence a device's inference execution environment. Bandwidth refers to the current network bandwidth between the device and the connected edge server or other edge devices. The connection is established via Wi-Fi, and the variable indicates the current transmission speed. This speed is used as an important parameter throughout the task scheduling process. Power represents the current flow and voltage of device. In wireless environments, it is used to calculate the remaining battery capacity, whereas in wired environments, it is used to monitor the power currently being supplied. The remaining power is also used as a key parameter in task scheduling strategies that aim to ensure system longevity.

5. Summary

The heterogeneity and resource constraints of edge devices can lead to degradation in the quality of AI services. To address this issue, this paper proposes a quantified resource unit called the Resource-Cell. A Resource-Cell is a metric that quantifies the actual performance of a device based on the computational workload and memory usage required during inference. This metric enables accurate prediction of inference performance when deploying AI models and allows heterogeneous devices to be compared using a single, unified metric.

A Resource-Cell is composed of the device's processing capabilities for four AI operations: GEMM, Convolution, Depthwise Convolution, and Attention. A device consists of multiple Resource-Cells, and the number of cells varies depending on its computational and memory architecture. Based on these Resource-Cells, this study constructs a Device Resource Table, whose elements serve as essential information for effective AI task scheduling and resource management in distributed environments.

Ⅳ. Multi-Dimensional Benchmark Suite

1. Taxonomy of AI Model Operator

The characteristics of an artificial intelligence (AI) model vary depending on the problem it is designed to solve. AI models were originally introduced to address problems that are not linearly separable. In the early stages, a primitive form of the Multi-Layer Perceptron (MLP) was proposed, and later, LeNet-5 in 1989 for handwritten digit recognition. Subsequently, after AlexNet archieved remarkable performance in the 2012 ILSVRC (ImageNet Large Scale Visual Recognition Challenge), AI models have rapidly evolved into diverse and sophisticated architectures. In this study, we classify the computational operations of such AI models and identify the key operations required for benchmarking [27]. Table 2 presents the fundamental operators along with the operators commonly used in AI models.

Table 2. AI Operator and Representative Model

Operator	Representative Model
Matrix Multiplication (GEMM)	Multi-layer Perceptron
Convolution (Conv.)	LeNet-5[28], AlexNet[29], ResNet[30]
Depthwise Conv.	MobileNet[31], EfficientNet[32]
Pointwise Conv.	ResNet, MobileNet, EfficientNet
Scaled Dot-Product Attention	Transformer[33], ViT[34]
Multi-Head Attention (MHA)	BERT[35], GPT[36]

2. Benchmark Suite

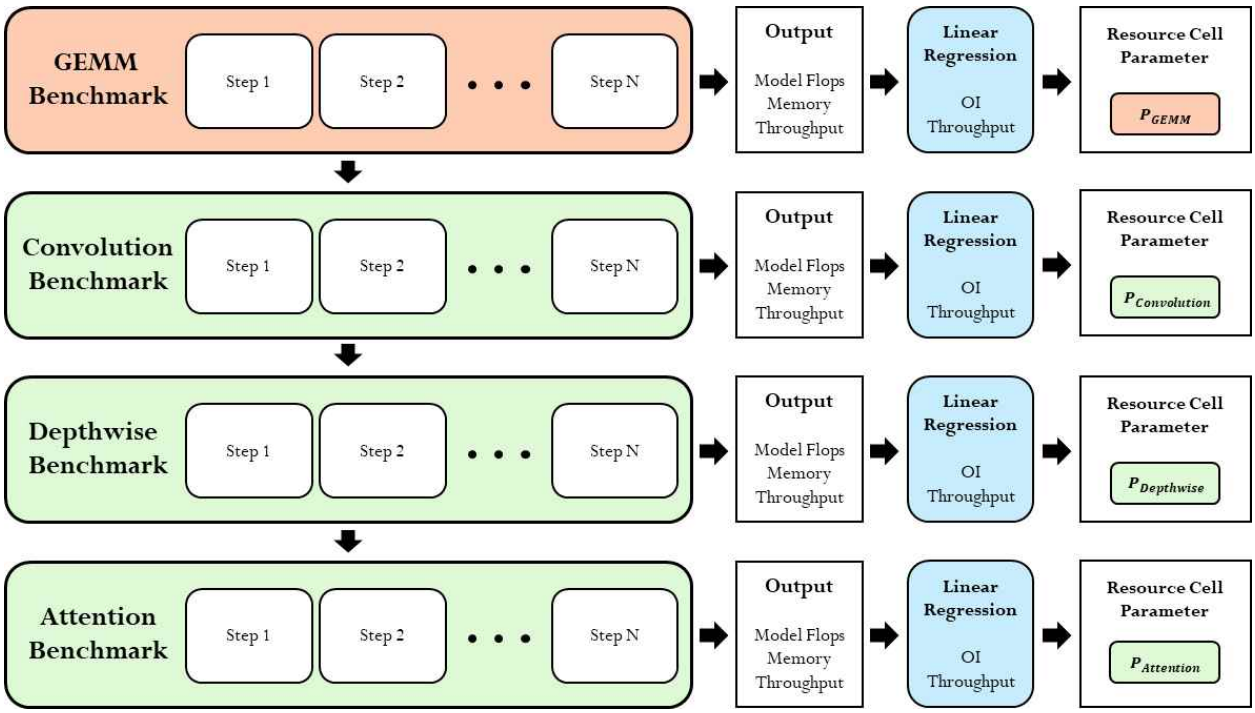

Figure 7. Overview of the Multi-Dimensional Benchmark Suite Pipeline

Figure 7 illustrates the benchmark suite designed based on these operators. Each benchmark is executed in a step-by-step manner, calculating the throughput and operational intensity (OI). After the benchmarking process is completed, a linear regression model is generated using the measured throughput and OI. This allows the performance of each operator to be estimated, producing the corresponding Resource-Cell parameters.

The diversity of artificial intelligence models cannot be evaluated solely by how fast a CPU processes floating-point operations. Because each model exhibits different computational loads and memory-access patterns, even models with similar FLOPs may demonstrate significantly different throughput depending on their structural characteristics. However, considering all possible operators during benchmarking increases complexity and leads to substantial execution time, which reduces practicality in dynamic edge environments. In this study, we focus on four representative operators that dominate the computational workload of modern AI models. The Resource-Cell benchmark is therefore constructed using Matrix Multiplication, Convolution, Depthwise Convolution, and Scaled Dot-Product Attention.

1) GEMM(General Matrix Multiplication)

Matrix Multiplication is the most fundamental type of operator in artificial intelligence models. Given matrices A and B, their inner product produces matrix C. This operation is compute-intensive and is therefore commonly used to measure floating-point operation performance.

$$C = A \times B, where\ A \in \mathbb{R}^{M \times K}, B \in \mathbb{R}^{K \times N}, C \in \mathbb{R}^{M \times N}, C_{i,j} = \sum_{k=1}^{K} A_{i,k} B_{k,j} \quad (1)$$

The extended version, known as GEMM (General Matrix Multiplication), does not simply compute the inner product of matrices but incorporates an additional matrix C to produce an output matrix C'. The extended formula is as follows.

$$C' = A \times B + C, where\ A \in \mathbb{R}^{M \times K}, B \in \mathbb{R}^{K \times N}, C \in \mathbb{R}^{M \times N}, C'_{i,j} = \sum_{k=1}^{K} A_{i,k} B_{k,j} + C_{i,j} \quad (2)$$

GEMM extends standard matrix multiplication by generating C', which incurs additional read and write operations and consequently increases memory access. Nevertheless, the total computational cost remains unchanged at $2MNK$ FLOPs.

2) Convolution

Convolution is a computational operation in which one of two continuous or discrete functions is flipped and shifted while computing their overlapping region. It is one of the representative operators used in CNN-based artificial intelligence models. A CNN processes image data through operations. In the benchmark used in this study, two types of convolution operations are employed.

(1) Standard Convolution

This represents the basic form of the 2D convolution operation. Given an input matrix and a kernel matrix, the kernel is slid across the input with a fixed stride to generate output values. This process is repeated over all input and output channels as required. The commonly used mathematical formulation is as follows:

$$Y \in \mathbb{R}^{B \times C_{out} \times H_{out} \times W_{out}}, where\ X \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}}, W \in \mathbb{R}^{C_{out} \times C_{in} \times K_H \times K_W} \quad (3)$$

$$Y_{b,o,i,j} = \sum_{c=1}^{C_{in}} \sum_{u=0}^{K_{H-1}} \sum_{v=0}^{K_{W-1}} X_{b,c,i+u,j+v} W_{o,c,u,v}$$

In Equation (3), X denotes the input feature map, represented as a two-dimensional vector, and W denotes the kernel weights, also represented as a two-dimensional tensor. Y represents the output feature map. B indicates the batch size processed at once, and C denotes the number of channels

(2) Depthwise Convolution

In a standard convolution, each output channel is generated by combining feature maps and kernels associated with all input channels. Therefore, as the number of input and output channels increases, both the computational cost and memory access grow significantly. In contrast, depthwise convolution assigns a single kernel to each input channel, eliminating the channel-combining process present in standard convolution. The operation can be formulated as follows:

$$Y \in \mathbb{R}^{B \times C_{out} \times H_{out} \times W_{out}},\ where\ X \in \mathbb{R}^{B \times C_{in} \times H_{in} \times W_{in}},\ W \in \mathbb{R}^{1 \times C_{in} \times K_H \times K_W} \quad (4)$$
$$Y_{b,o,i,j} = \sum_{u=0}^{K_{H-1}} \sum_{v=0}^{K_{W-1}} X_{b,c,i+u,j+v} W_{c,0,u,v}$$

Although the input and output feature maps share the same spatial dimensions, the model becomes lightweight because only one kernel weight is assigned per input channel.

(3) Attention

Attention is a core component of Transformer-based models, and in this paper, the operator used for benchmark is the Scaled Dot-Product Attention. The input tokens are projected into three feature vectors: Query (Q), Key (K), and Value (V). The similarity between the Queries and Keys is computed using dot products, and the resulting scores are scaled by the square root of the dimensionality to obtain the attention weights. These weights are then applied to the Values to produce the output. Multi-Head Attention(MHA) is an extension of this process that parallelizes multiple attention heads.

$$Attention(Q, K, V) = softmax(\frac{QK^T}{\sqrt{d_k}})V \quad (6)$$

3. Implementation

1) Device Setup

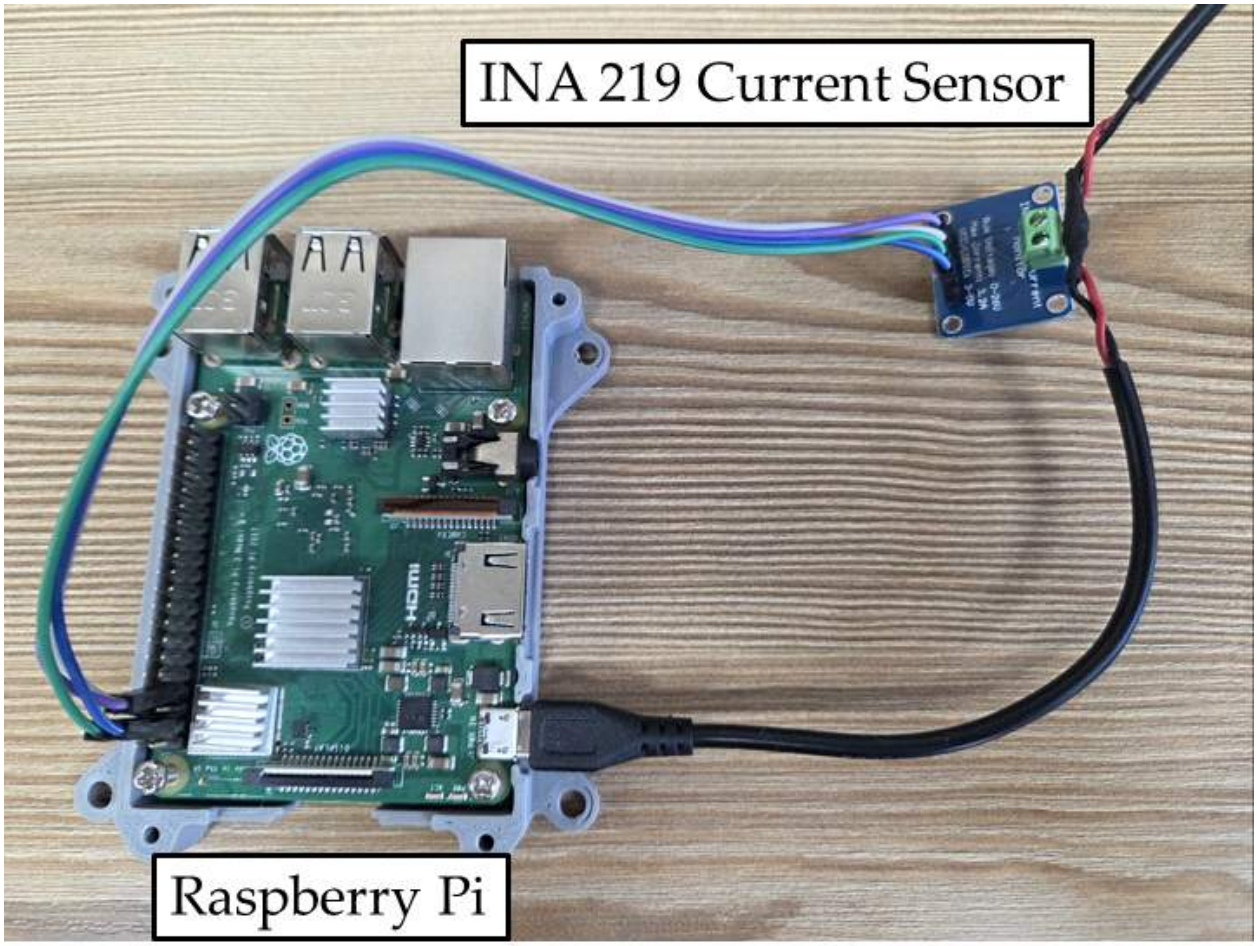

Figure 8. Raspberry Pi with Current Sensor

For benchmarking edge devices, this study utilized the Raspberry Pi, one of the most representative platforms in the Internet of Things (IoT) domain.

The Raspberry Pi is an educational single-board computer developed in the United Kingdom. It is a Linux-based single-board computer equipped with an ARM-based processor, and due to its versatility, it is widely used as an edge device across various application domains. Considering this versatility, the Raspberry Pi was selected as the testbed for this study.

The objective of this research is to measure performance in heterogeneous environments, quantify the results, and evaluate their accuracy.

Figure 8 shows the connection configuration of the INA219 current sensor and the Raspberry Pi used in the experiment. The sensor is physically connected to the power

supply line of the Raspberry Pi, enabling real-time measurement of power consumption. The INA219 sensor calculates the current by measuring the voltage across a shunt resistor and using its known resistance value [37].

However, when architectures differ entirely, it becomes difficult to control experimental variables. Therefore, although hardware performance varies across devices, the CPU and memory architectures were kept consistent. Accordingly, three types of Raspberry Pi devices – Raspberry Pi 3B+, Raspberry Pi 4B, and Raspberry Pi 5B – were used in the experiments. Although the release dates, CPU capabilities, and memory performance of these models differ, they share a common architectural foundation.

The specifications of the Raspberry Pi devices used in the implementation are summarized in Table 3.

Table 3. Raspberry Pi Series specification [38-40]

Model	Processor	Memory
Raspberry Pi 3 B+	1.4GHz ARM Cortex-A53 MP4	1 GB LPDDR2-500 SDRAM
Raspberry Pi 4 B	1.5GHz ARM Cortex-A72 MP4	2 GB LPDDR4-3200 SDRAM
Raspberry Pi 5 B	2.4GHz ARM Cortex-A76 MP4	8 GB LPDDR4X-4266 SDRAM

2) Benchmark Suite Parameter

The AI workloads were implemented using PyTorch. The Resource-Cell benchmark is designed with a focus on reproducibility. While workloads built with C-based compilers are highly optimized, they have the drawback of being difficult for users to implement manually. PyTorch, on the other hand, is widely used in academia for model verification. Because it supports a wide variety of models through its extensive library ecosystem, benchmarks can be easily reproduced in any environment where PyTorch is available. From a performance perspective, PyTorch is implemented in C++ and is optimized for both model training and inference. In this study, Python 3.9 and PyTorch 2.0 were used.

The benchmark progressively scales the model by increasing both the computational workload and memory access volume, allowing performance variations caused by memory bottlenecks such as cache misses. The parameters of the model are defined as follows:

Table 4. Benchmark Model Parameter

Benchmark	MIN	MAX	Nonlinear
GEMM	M=256, K=256, N=256	M=1024, K=1024, N=1024	ReLU Softmax
Convolution	B=1, Cin=6, Cout=12, H=224,W=224,K=3	B=1, Cin=24, Cout=48, H=224,W=224,K=3	ReLU, BatchNorm, AveragePooling
Depthwise	B=1, C=8, H=64, W=64, K=3	B=1, C=96, H=256, W=256, K=3	ReLU, BatchNorm, AveragePooling
Attention	Token=160, Head=4, HiddenDimension=64	Token=640, Head=8, HiddenDimension=64	LayerNorm

Table 5. Flops and Memory Usage per Benchmark Step

Model	1		2		…	9		10	
GEMM	33.5MFlops	1MB	79.3MFlops	1.86MB		1,655.8MFlops	14MB	2,147.5MFlops	16.8MB
Convolution	65MFlops	3.6MB	115.6MFlops	4.8MB	…	874.2MFlops	13MB	1,040.5MFlops	14.5MB
Depthwise	1.3MFlops	0.4MB	2.3MFlops	1MB		85.5MFlops	38MB	113.3MFlops	50.3MB
Attention	110.5MFlops	2MB	157.7MFlops	3.6MB		1,908.3MFlops	33MB	2,146.6MFlops	38.5MB

4. Result

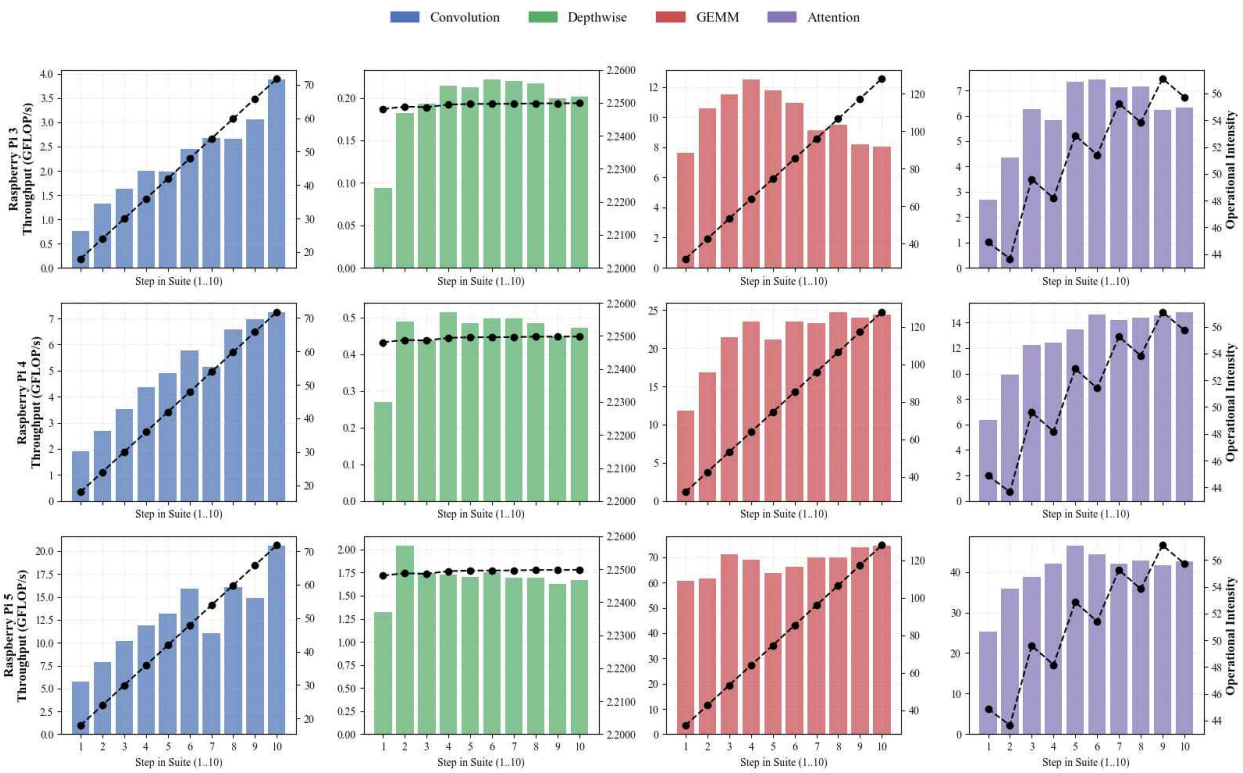

Figure 9. Results of the Benchmark Suite on Each Device

The Resource-Cell is computed by considering both CPU performance and memory bandwidth. Figure 9 shows the measured processing performance across three versions of Raspberry Pi devices. Each benchmark consists of ten steps in which the computational load and memory usage gradually increase. The x-axis represents the incremental steps, while the y-axis indicates the throughput. The rate of increase in computation and memory usage varies depending on the benchmark model, as summarized in Table 5.

Four benchmarks were executed on each device, and the results represent the averages over 15 runs. Each set comprises 10 indices, and the values are averaged within each set to observe stability and cache effects during repeated executions of the same operation. Each plot shows the throughput and OI across the incremental steps.

Depthwise convolution is a memory-intensive operation. Therefore, as the model size increases, both computation and memory access grow proportionally, resulting in an almost constant OI across all steps. In contrast, the OI of Convolution and GEMM increases linearly with the stage index. For Convolution, throughput increases linearly with OI. GEMM reaches its peak performance around Step 3 and tends to either decline or remain stable thereafter. The Attention operator exhibits a slight decrease followed by an increase in OI due to its structural characteristics.

Table 6 presents the computed Resource-Cell results. For each benchmark, a linear regression model was trained using operational intensity (OI) and throughput, and the number of cells was obtained by inputting a target OI into the model. The target OI was set to the average OI of each benchmark. If the target OI is set too low, the device performance may be underestimated. Conversely, setting it too high may lead to an overestimation of performance. Therefore, the average OI observed in each benchmark was used to derive the Resource-Cell values.

Table 6. Resource-Cell Values Computed for Each Operator

Operator	Average OI	Raspberry Pi 3	Raspberry Pi 4	Raspberry Pi 5
GEMM	79.99	998.21	2146.84	6814.79
Convolution	44.91	224.22	452.05	1187.91
Depthwise	2.25	19.55	45.91	169.76
Attention	51.25	607.286	2146.54	4016.55

5. Summary

In this chapter, we classify the core computational operators that representative AI models (Operator Taxonomy) and propose a multi dimensional benchmark suite based on these operators. Because AI models exhibit substantial variation in computational load and memory access patterns depending on their structural characteristics, performance cannot be accurately assessed using only FLOPs. Therefore, this study focuses on four representative operators that dominate the computational workload of modern AI models:

- GEMM,
- Convolution,
- Depthwise Convolution,
- Scaled Dot Product Attention.

To implement the multi dimensional benchmark suite, we used the Raspberry Pi series, which represents typical edge devices, and employed the INA219 current sensor to analyze the power efficiency of the computed Resource Cells. All benchmark models were implemented using PyTorch, a Python based framework. The benchmark consists of ten stages, and the results from these stages are used to train a linear regression model for Resource-Cell computation. By inputting the average operational intensity (OI) of each benchmark into the trained model, the Resource-Cell values for each operator on each device were derived.

V. Evaluation

1. Performance Prediction Accuracy

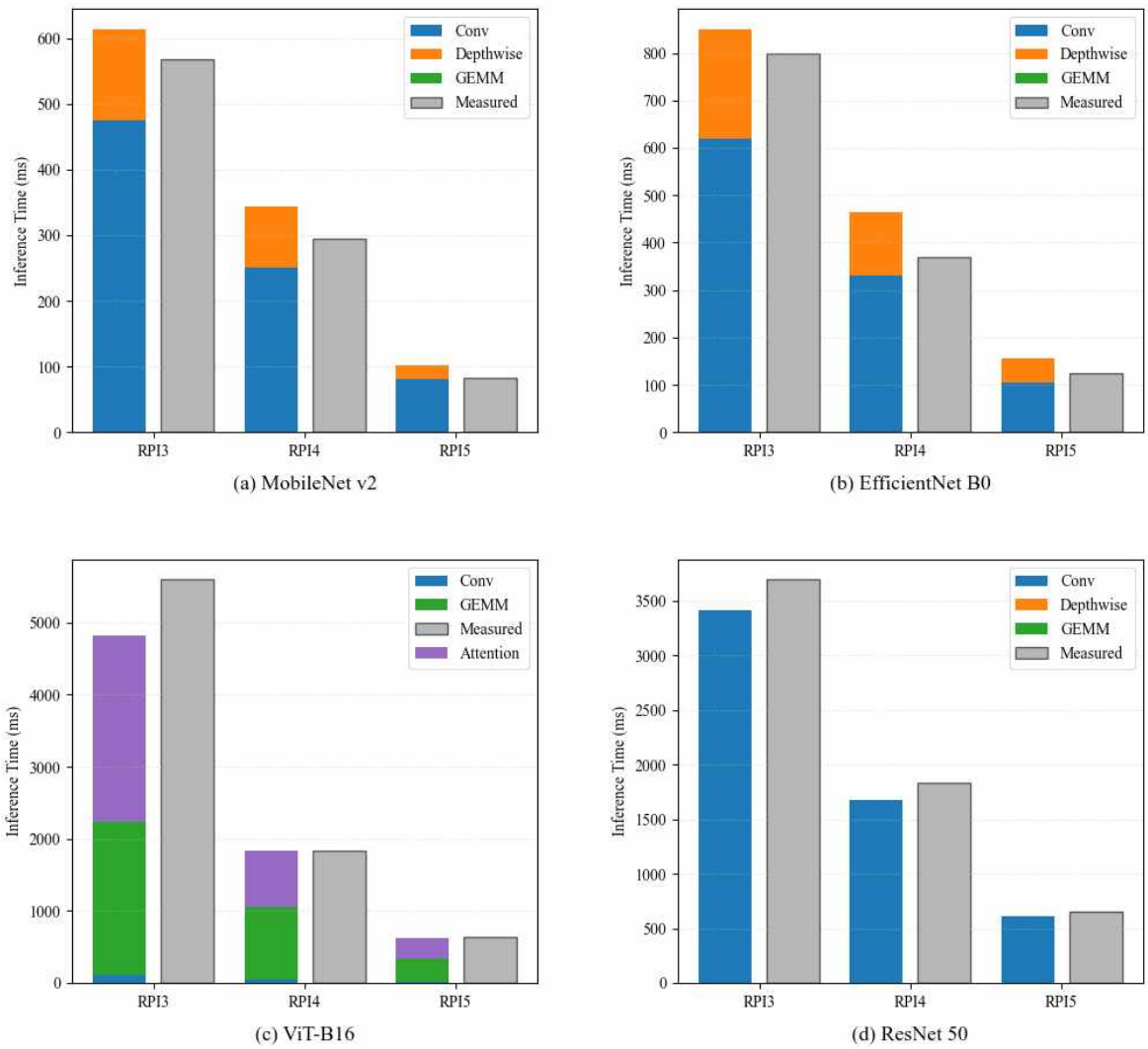

Figure 10. Inference Time Prediction in AI Models

Figure 10 presents a comparison between the Resource-Cell-based predictions and the actual measured inference time. The AI models used in this study are ImageNet-based vision models. Because these models extensively utilize the four types of operations proposed in this work, they provide an appropriate basis for comparing inference performance using the Resource-Cell metric. The performance characteristics across models, as well as the generational differences among devices, can be interpreted as follows.

1) MobileNet

MobileNet V2 is the smallest among the four models, and due to its architectural characteristics, depthwise convolutions account for a significant portion of the total computation time. On the RPi 3, the combined inference time of convolution and depthwise operations is predicted to be around 600 ms, showing a trend consistent to the measured results. The RPi 4 shows an error of 12.7%, while the RPi 5 exhibits an error of 22.4%. Although the absolute prediction error is small, the inference time itself is short, resulting in an average relative error of approximately 13%.

2) EffecientNet

On the RPi 3, the combined latency of convolution and depthwise operations is approximately 850 ms, which is higher than that of MobileNet V2. Although the relative proportion of operations is not significantly different from MobileNet V2, the increased model complexity and computational load result in a longer inference time. The RPi 4 shows an error of 15.0%, while the RPi 5 exhibits an error of 6.8%. Overall, the prediction patterns are similar to those observed for MobileNet, with an average error of approximately 9%.

3) Vision Transformer

ViT-B/16 contains only a small number of convolutional operations, and its overall computation is predominantly dominated by self-attention and large-scale GEMM operations. Although self-attention was originally introduced in language models, it has since been extended and widely adopted in modern vision architectures.

On the RPi 3, the combined latency of attention and GEMM operations reaches approximately 4800 ms. Although ViT exhibits the longest inference time among the evaluated models, its total computational workload is about 45.5 times larger than that of EfficientNet. Because ViT heavily relies on GEMM, the operation with the highest computational throughput, the performance improvement on the RPi 5 is substantially larger compared to the other vision models. The average prediction error is approximately 14%.

4) ResNet

ResNet-50 is dominated almost entirely by convolution operations, which account for nearly all of its total latency. As a result, unlike the other models, its inference time tends to be consistently under-predicted. The average inference error is approximately 7%.

2. Power Consumption

The power consumption of a Resource-Cell is modeled as the sum of the device's baseline idle power and the additional power proportional to the activation level and utilization duration of the Resource-Cells. We approximate this behavior by assuming that the increase in Resource-Cell utilization is linearly correlated with the rise in CPU utilization. In other words, higher CPU utilization corresponds to a greater number of concurrently active Resource-Cells, which in turn leads to increased power consumption. Accordingly, the power consumption model for Resource- Cells is defined as follows.

$$Power\ Consumption = (I + P_{rc} \times t) \quad (7)$$

In Equation (7), I denotes the baseline power consumption when the device is in an idle state. P_{rc} represents the power consumed by a single active Resource Cell, is the number of activated Resource Cells, and t denotes the activation duration.

In this step, the number of active Resource Cells at each moment was estimated using the device-specific Resource-Cells derived in Section Ⅳ.5 and the measured CPU utilization. The power consumption was then measured while gradually increasing the Resource-Cell activation level. The relationship between the measured power consumption and CPU utilization was modeled using linear regression. The loss function was computed based on the Mean Square Error (MSE).

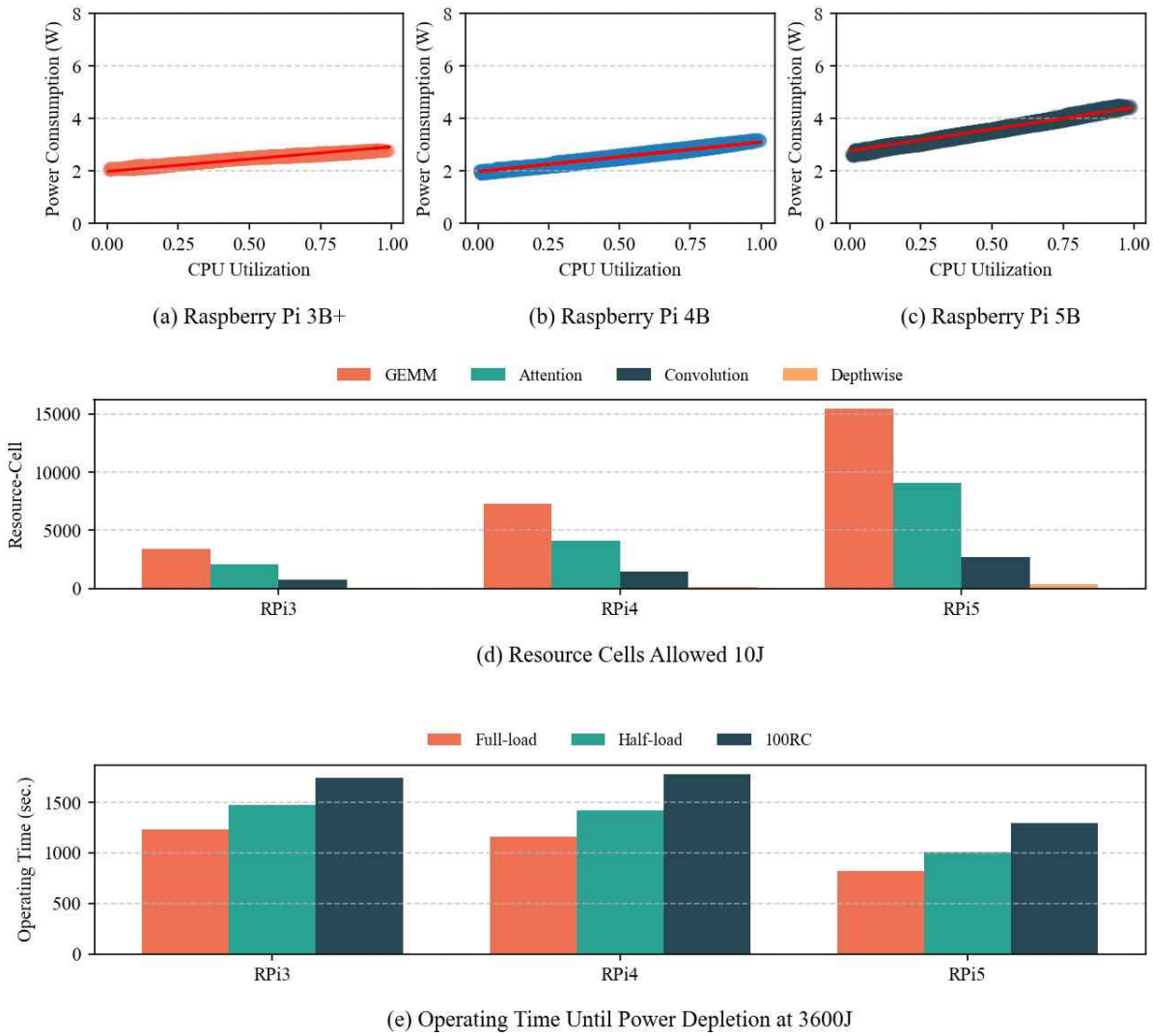

Figure 11. Power Consumption Model and Performance Evaluation

Figure 11 presents the measured power consumption and regression analysis results for three Raspberry Pi devices (3B+, 4B, and 5B) under varying levels of Resource-Cell activation. The figure illustrates the practical performance characteristics under power-constrained conditions.

In Figure 11, (a) - (c) show linear regression models that estimate the power consumption of Resource-Cells as a function of CPU utilization. The x-axis represents CPU utilization (0 - 1), and the y-axis denotes the power consumed per second. The results show that power consumption increases linearly with CPU utilization across all devices. Among them, the RPi 5 exhibits the highest rate of increase, indicating superior performance but also higher power draw, whereas the RPi 3 shows both low performance and low power characteristics.

(d) analyzes the efficiency of Resource-Cells with respect to available energy. It

shows the number of Resource-Cells that can remain active within an energy budget of 10 J. The x-axis represents the four operation types constituting a Resource-Cell across each device, and the y-axis denotes the number of activatable Resource-Cells. Under constrained energy conditions, RPi 5 achieves higher effective performance due to its superior computational efficiency.

(e) illustrates the sustained operating time as a function of Resource-Cell activation under a 3600 J energy budget. Each device is evaluated in three states: fully activated, half activated, and intermittently activated. Full activation corresponds to executing a large workload continuously with no idle time, and in this case the RPi 3 shows the longest survival time, although its processing performance is approximately eight times lower than that of the RPi 5. However, in scenarios dominated by idle intervals, the RPi 3 demonstrates significantly longer uptime, indicating that it offers higher energy efficiency when workloads are sparse or intermittent.

3. Task Distribution Evaluation

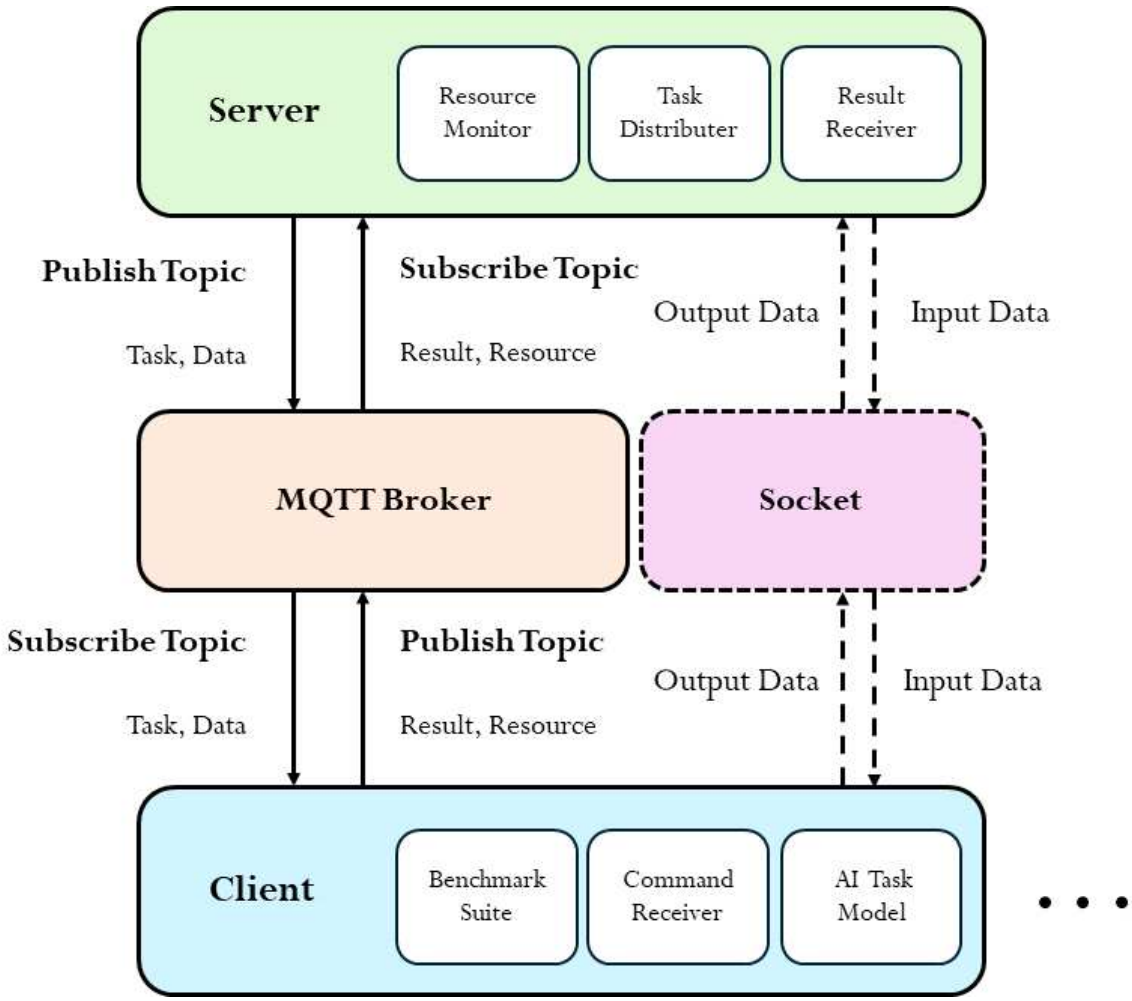

Figure 12. MQTT-based Distributed Processing Architecture with Resource-Cell

In this study, we implemented an MQTT-based communication framework to enable efficient distributed inference in edge environments. Figure 12 provides a overview of the entire system architecture, illustrating the roles of each component and the flow of communication among them. The server centrally publishes task execution commands and related metadata to the clients via MQTT messages, while subscribing to the resource status information and task results that clients periodically publish. Conversely, each client subscribes to the server's task commands and metadata, and periodically publishes its resource status and task execution results.

Large data items such as input tensors for AI models or intermediate inference outputs are exchanged through socket communication rather than MQTT, with MQTT messages carrying only the associated metadata. The server continuously monitors the resource status of connected client devices in real time and distributes tasks to

appropriate clients based on this information. It also collects the task results returned by each client and utilizes them for subsequent processing.

Each client executes two types of AI workloads. First, benchmark suite is used to compute the Resource-Cells. Second, clients perform actual AI inference workloads based on the tasks assigned by the server.

Client ID	IP	Task	GEMM	CONV	DEPTH	ATTN	Bandwidth	Memory	Battery	Timestamp
rpi4	192.168.101.2	Ready	2780	641	47	1995	0.3	1312.6	100.0	2025-11-13 01:33:43
rpi3	192.168.101.12	Ready	875	312	20	1056	0.7	475.8	100.0	2025-11-13 01:33:44
rpi5	192.168.101.9	Ready	6149	1426	144	4323	1.3	3382.5	100.0	2025-11-13 01:33:44

Figure 13. Execution Screen of Resource Monitoring on the Server

Figure 13 is an example of the resource table displayed on the server, which monitors the resource status of each client device. The table includes the Client ID for device identification, the IP address for communication, and the current task status. The number of available Resource-Cells is represented separately for the four operators-GEMM, CONV, DEPTH, and ATTN. In addition, the table provides information such as network bandwidth, memory capacity, and remaining battery capacity. In this particular example, the devices are powered through a wired connection, and therefore the battery field shows a default value.

1) Load Balance Evaluation at Distributed Processing

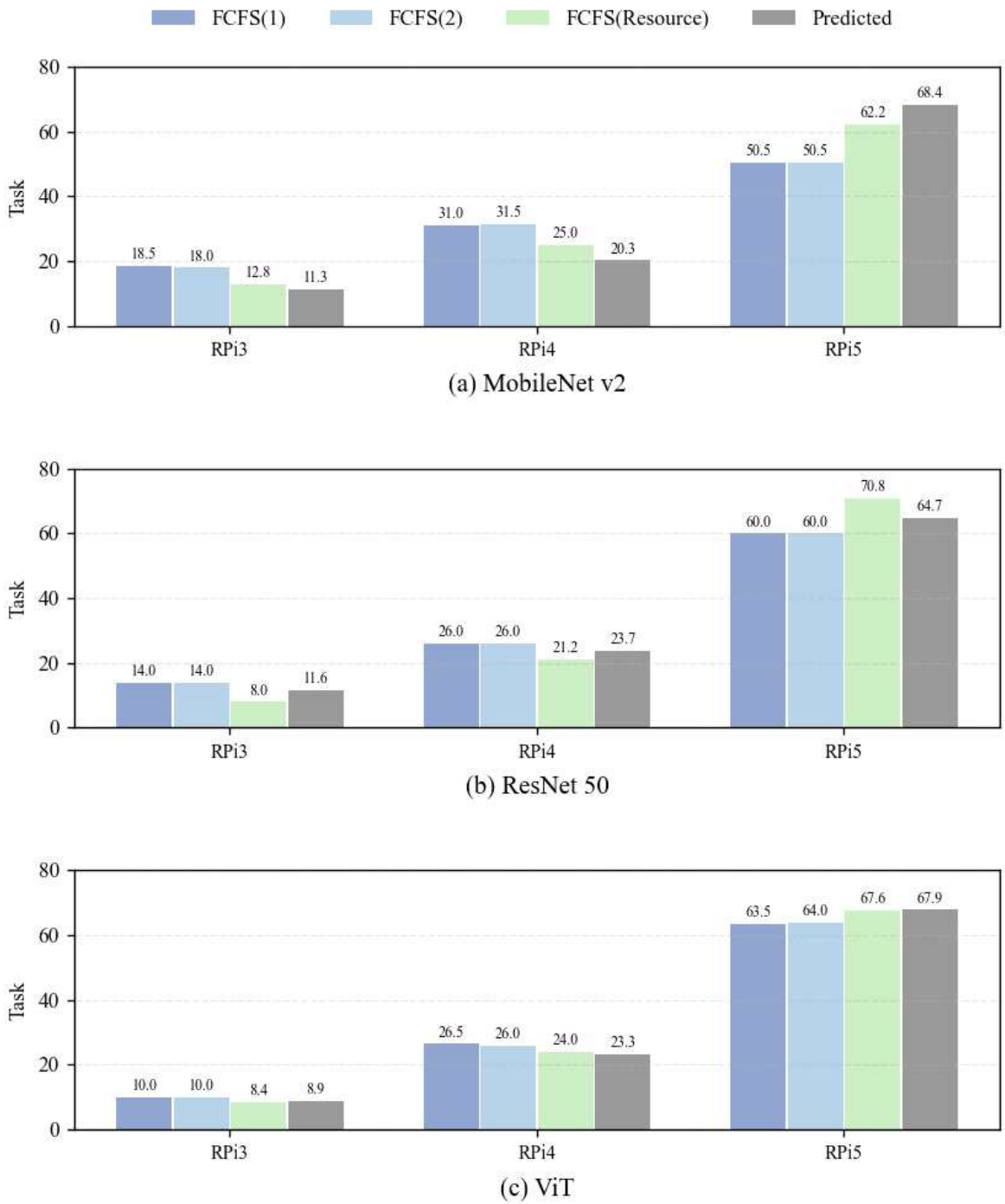

Figure 14. Inference Counts per Device Across Task Distribution Strategies

Figure 14 presents the total number of inference tasks processed by each device (RPi 3, 4, and 5), illustrating the distributed processing results for three AI models: MobileNet-V2, ResNet-50, and ViT. Each distributed execution result includes a comparison between the FCFS-based task distribution strategy and the throughput predicted using the Resource-Cell-based estimation model.

FCFS(1) transmits one data tensor per request. FCFS(2) transmits two data tensors per request. FCFS(Resource) allocates the amount of data transmitted in a single request in proportion to the Resource-Cells of each device.

Overall, the analysis shows that resource-based task distribution provides limited effectiveness for MobileNet, due to its small model size and the resulting communication overhead and idle latency. In contrast, for large models such as ViT—whose computational efficiency is high and communication overhead is relatively small—the resource-based task distribution strategy offers clear advantages and yields more effective performance scaling.

2) Turnaround Time Evaluation

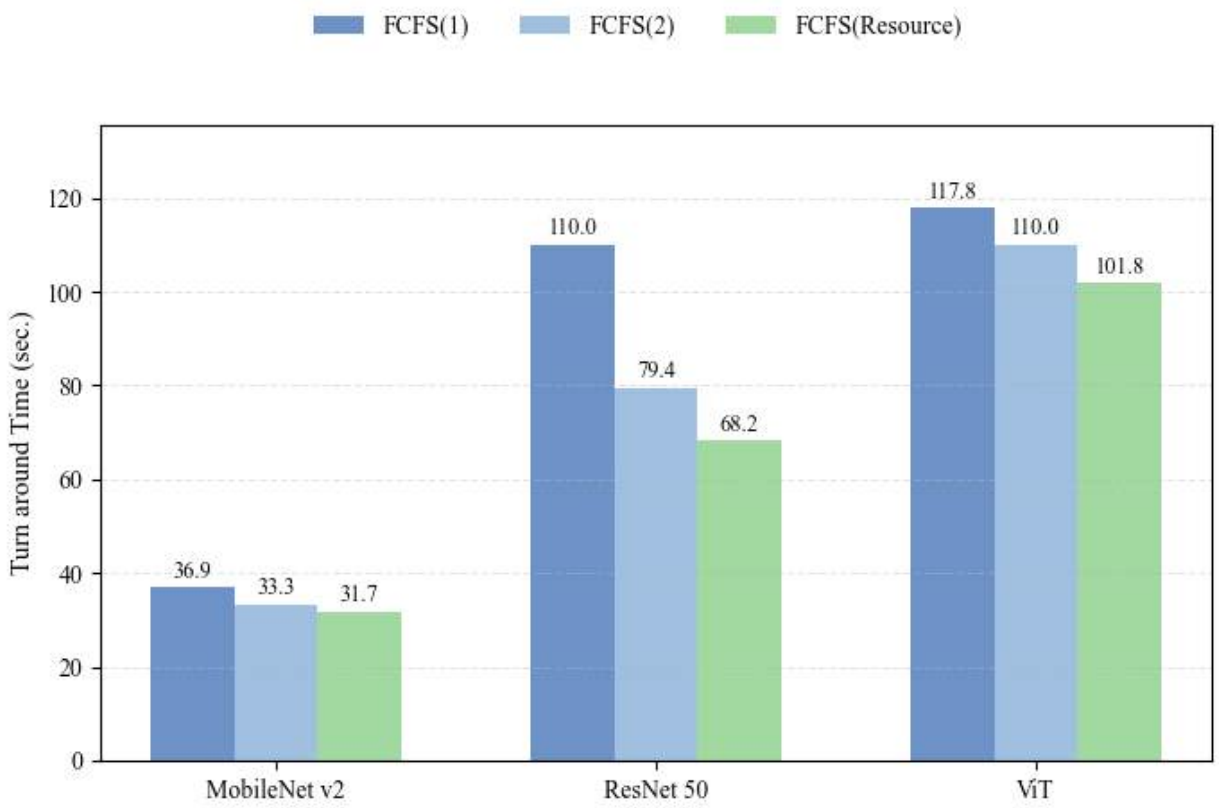

Figure 15. Comparison of Turnaround Time Across Task Distribution Strategies

Figure 15 presents the total turnaround time of MobileNetV2, ResNet-50, and ViT under three scheduling strategies: FCFS(1), FCFS(2), and FCFS(Resource). Overall, the FCFS(Resource) strategy consistently achieves the lowest latency across all models, demonstrating superior performance compared to the conventional FCFS-based approaches. For MobileNetV2, FCFS(Resource) achieves a 14.1% reduction in latency compared to FCFS(1) and operates 4.7% faster than FCFS(2). For ResNet-50, both FCFS(2) and the resource-based strategy show more than a 27% improvement over FCFS(1), while the resource-based scheduling achieves a 38% reduction in turnaround time. ViT, the most computationally intensive model among the three, benefits significantly from distributed processing and exhibits a 13.6% improvement over FCFS(1). Compared to ResNet-50, however, the relative improvement is smaller, indicating that ViT experiences computation-bound bottlenecks rather than communication overhead due to its highly compute-intensive structure.

4. Summary

This chapter presents a comprehensive evaluation of the effectiveness and performance of the proposed Resource-Cell structure. A Resource-Cell represents the actual processing capabilities of a device for AI operations and is used to predict inference performance. To assess the accuracy of these predictions, four vision models—MobileNet-V2, EfficientNet, ViT-B/16, and ResNet-50 were profiled to measure the computational cost of their constituent operators. Using the Resource-Cell configurations of each device, inference times were estimated and compared with measured values. The results show an overall prediction accuracy of approximately 90%, demonstrating that the Resource-Cell metric effectively reflects real inference performance.

To analyze the power efficiency of Resource-Cells, power consumption was measured across varying CPU utilization levels, and a linear regression model was constructed. Two evaluation scenarios were applied to compare the power characteristics of different devices. While the Raspberry Pi 5 exhibited superior computational efficiency, the Raspberry Pi 3 proved more advantageous in scenarios involving long-term intermittent workloads due to its lower idle power consumption.

To evaluate the Resource-Cell-based task distribution strategy, an MQTT-based distributed processing framework was implemented. The scheduling strategy was built on FCFS and compared under two conditions: equal data allocation and Resource-Cell-based proportional allocation. Distributed processing performance was assessed using load balancing and turnaround time as metrics. Depending on the model, the Resource-Cell-based strategy achieved performance improvements ranging from 14% to 38%, confirming its effectiveness in heterogeneous edge environments.

Ⅵ. Conclusion

Edge computing environments face inherent challenges arising from resource constraints, device heterogeneity, and energy dependency. To address these practical limitations, this paper proposes the Resource-Cell structure, a quantified unit that represents the computational capabilities of edge devices. Each Resource-Cell expresses the device's performance to execute the key operations required for AI inference.

In this paper, we employ the Multi-Dimensional Benchmark Suite consisting of GEMM, Convolution, Depthwise Convolution, and Attention to determine the Resource-Cell capacity of each device. Each benchmark includes ten stages in which the computational workload and memory usage gradually increase, allowing throughput and OI to be measured. A linear regression model is then trained using the measured throughput and OI, and the number of Resource-Cells is estimated by applying the average OI of each benchmark.

To evaluate the accuracy of the proposed metric, we assess its predictive capability using representative AI models including MobileNet V2, ResNet-50, EfficientNet, and ViT-16. The Resource-Cell metric achieves an average prediction accuracy of 89.7 %, demonstrating its effectiveness in estimating device level inference performance. Furthermore, we analyze power consumption using device specific Resource-Cell profiles and provide deployment strategies tailored to device conditions.

Finally, in distributed inference scenarios, the Resource-Cell based task scheduling strategy outperforms conventional FCFS methods, reducing total processing time by an average of 15.4 % and by up to 38 % in the best case. These results show that quantifying device capabilities through Resource-Cells is beneficial for performance comparison and distributed task allocation in edge environments. Future work includes expanding the benchmark dimensions and developing more fine grained resource aware scheduling strategies.

References

1. J. Aljbour, T. Wilson, and P. Patel, “Powering Intelligence: Analyzing Artificial Intelligence and Data Center Energy Consumption,” *EPRI White Paper No. 3002028905*, May. 2024.
2. W. Shi, J. Cao, Q. Zhang, Y. Li, and L. Xu, “Edge computing: Vision and challenges,” *IEEE Internet of Things Journal*, Vol. 3, No. 5, pp. 637 - 646, Oct. 2016.
3. M. Satyanarayanan, “The Emergence of Edge Computing,” *Computer*, Vol. 50, No. 1, pp. 30 - 39, Jan. 2017
4. T. Taleb, K. Samdanis, B. Mada, H. Flinck, S. Dutta, and D. Sabella, “On Multi-Access Edge Computing: A Survey of the Emerging 5G Network Edge Cloud Architecture and Orchestration,” *IEEE Communications Surveys & Tutorials*, Vol. 19, No. 3, pp. 1657 - 1681, May. 2017.
5. W. Yu, F. Liang, X. He, W. G. Hatcher, C. Lu, J. Lin, and X. Yang, “A Survey on the Edge Computing for the Internet of Things,” *IEEE Access*, Vol. 6, pp. 6900 - 6919, Nov. 2017.
6. N. Abbas, Y. Zhang, A. Taherkordi, and T. Skeie, “Mobile Edge Computing: A Survey,” *IEEE Internet of Things Journal*, Vol. 5, No. 1, pp. 450 - 465, Feb. 2018.
7. J. Chen and X. Ran, “Deep Learning with Edge Computing: A Review,” *Proceedings of the IEEE*, Vol. 107, No. 8, pp. 1655 - 1674, Aug. 2019.
8. Singh, Raghubir, and Sukhpal Singh Gill. “Edge AI: A Survey,” *Internet of Things and Cyber-Physical Systems*, Vol. 3, pp. 71-92, Mar. 2023.
9. S. S. Gill et al., “Edge AI: A Taxonomy, Systematic Review and Future Directions,” *Cluster Computing*, Vol. 28, No. 1, pp. 1 - 53, Oct. 2024.
10. Sander. Jacob, et al. “On Accelerating Edge AI: Optimizing resource-constrained environments.” *arXiv preprint arXiv:2501.15014,* Jan. 2025.
11. Chitty-Venkata, Krishna Teja, and Arun K. Somani. “Neural architecture search survey: A hardware perspective,” *ACM Computing Surveys,* Vol. 55(4), No. 78, pp. 1-36, Nov. 2022.
12. Benmeziane, Hadjer, et al. “Hardware-Aware Neural Architecture Search: Survey and Taxonomy,” *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence (IJCAI-21)*, Vol. 2021, pp 4322-4329, 2021.
13. Thomas Elsken, Jan Hendrik Metzen, and Frank Hutter, “Neural architecture search: A survey,” *Journal of Machine Learning Research*, Vo1. 20, No. 55, pp. 1-21, 2019.
14. X. Wang and W. Jia, “Optimizing Edge AI: A Comprehensive Survey on Data, Model, and System Strategies,” *arXiv preprint*, arXiv:2501.03265, Jan. 2025.
15. Duan Sijing, et al., “Distributed Artificial Intelligence Empowered by End-Edge-Cloud Computing: A survey,” *IEEE Communications Surveys & Tutorials*, Vol. 25, No. 1 pp. 591-624, Nov. 2022.
16. Y. Kang et al., “Neurosurgeon: Collaborative Intelligence Between the Cloud and Mobile Edge,” *ACM SIGARCH Computer Architecture News*, Vol. 45, No. 1, pp. 615 - 629, Apr. 2017.

17. M. Zawish, L. Abraham, K. Dev, and S. Davy, “Towards Resource-aware DNN Partitioning for Edge Devices with Heterogeneous Resources,” *Proc.* , pp. 5649 - 5655, 2022.
18. Z. Zhao, K. M. Barijough, and A. Gerstlauer, “Deepthings: Distributed Adaptive Deep Learning Inference on Resource-Constrained IoT Edge Clusters,” *IEEE Transaction Computer-Aided Design of Integrated Circuits and Systems*, Vol. 37, No. 11, pp. 2348 - 2359, Nov. 2018.
19. A. A. Ismail, N. E. Khalifa, and R. A. El-Khoribi, “A Survey on Resource Scheduling Approaches in Multi-Access Edge Computing Environment: A Deep Reinforcement Learning Study,” *Cluster Computing*, Vol. 28, No. 184, Jan. 2025.
20. M. A. Hasanpour, M. Kirkegaard, and X. Fafoutis, “EdgeMark: An Automation and Benchmarking System for Embedded Artificial Intelligence Tools,” *arXiv preprint*, arXiv:2502.01700, Feb. 2025.
21. v. Kistowski, Jóakim, et al. “How to build a benchmark,” *Proc. Proceedings of the 6th ACM/SPEC international conference on performance engineering,* pp. 333-336, Jan. 2015.
22. V. J. Reddi et al., “MLPerf Inference Benchmark,” *Proc. 2020 ACM/IEEE 47th Annual International Symposium on Computer Architecture (ISCA)*, pp. 446 - 459, 2020.
23. A. Ignatov et al., “AI Benchmark: All About Deep Learning on Smartphones in 2019,” *Proc. IEEE/CVF International Conference Computer Vision Workshops (ICCVW)*, pp. 3617 - 3635. 2019.
24. TensorFlow. *TensorFlow Lite for Microcontrollers*. [Online]. Available: https://www.tensorflow.org/lite/microcontrollers. (accessed 2025, Nov, 1)
25. S. Williams, A. Waterman, and D. Patterson, “Roofline: An Insightful Visual Performance Model for Multicore Architectures,” *Communications of the ACM*, Vol. 52, No. 4, pp. 65 - 76, Apr. 2009.
26. A. Reuel et al., “BetterBench: Assessing AI Benchmarks, Uncovering Issues, and Establishing Best Practices,” *Proc. Advances in Neural Information Processing Systems 37 (NeurIPS 2024),* 2024.
27. Vivienne Sze, et al. “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE*, Vol. 105, No. 12, pp. 2295-2329, Nov. 2017.
28. Yann LeCun, et al. “Gradient-based Learning Applied to Document Recognition,” *Proceedings of the IEEE*, Vol. 86, No. 11, pp.2278-2324, Nov. 1998.
29. Alex Krizhevsky, Ilya Sutskever and Geoffrey E. Hinton, “Imagenet Classification with Deep Convolutional Neural Networks,” *Advances in neural information processing systems* 25, 2012.
30. Vivienne Sze, et al. “Efficient Processing of Deep Neural Networks: A Tutorial and Survey,” *Proceedings of the IEEE*, Vol. 105, No.12, pp. 2295-2329, Nov. 2017.
31. Kaiming He, et al. “Deep Residual Learning for Image Recognition,” *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition,* pp. 770-778, 2016.
32. Andrew G. Howard, et al. “MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications,” *arXiv preprint arXiv:1704.04861*, Apr. 2017.

33. Tan, Mingxing, and Quoc Le, “EfficientNet: Rethinking Model Scaling for Convolutional Neural Networks,” *Proceedings of the 36th International Conference on Machine Learning,* Vol. 97, pp. 6105-6114, 2019.
34. Ashish Vaswani, et al. “Attention is all you need,” *Advances in Neural Information Processing Systems* 30, 2017.
35. Alexey Dosovitskiy, et al. “An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale,” *arXiv preprint arXiv:2010.11929v2*, Jun. 2021.
36. Jacob Devlin, et al. "Bert: Pre-training of deep bidirectional transformers for language understanding." *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies,* Vol. 1, pp. 4171-4186, 2019.
37. Texas Instruments. *INA219 Precision Power Monitor*. [Online]. Available: https://www.ti.com/product/INA219 (accessed Nov. 15, 2025).
38. Raspberry Pi 5. *Raspiberry Pi 5 Tech specs,*[Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-5. (accessed Nov. 15, 2025).
39. Raspberry Pi 4. *Raspiberry Pi 4 Tech specs,*[Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-4-model-b/specifications. (accessed Nov. 15, 2025).
40. Raspberry Pi 3. *Raspiberry Pi 3B+ Tech specs,*[Online]. Available: https://www.raspberrypi.com/products/raspberry-pi-3-model-ab-plus/. (accessed Nov. 15, 2025).

엣지 컴퓨팅에서 분산 인공지능을 위한 자원 구조 연구

권 용 훈

강원대학교 대학원 컴퓨터정보통신공학과

초 록

클라우드 컴퓨팅의 발전은 인공지능 서비스를 제공하는데 큰 역할을 했다. 하지만 인공지능 서비스 수요 증가는 중앙 집중형 클라우드 서버의 에너지 소비와 지연 문제를 심화시키고 있다. 이를 해결하기 위해 엣지 컴퓨팅이 제안되었다. 엣지 컴퓨팅은 네트워크의 말단에서 데이터를 처리하여 빠르고, 데이터가 지역적으로 처리되기 때문에 보안에 유리하다. 하지만 엣지 컴퓨팅의 자원 제약성과 이기종성은 다양한 인공지능 서비스를 제공하는데 도전과제가 된다. 본 논문은 이러한 문제를 해결하기 위해 엣지 컴퓨팅 환경에 적합한 자원 구조를 제안한다. 리소스 셀은 디바이스가 인공지능 워크로드를 수행하는데 필요한 자원을 논리적으로 정량화한 단위이다. 리소스 셀은 디바이스의 성능을 이론적인 수치가 아니라 실제 성능하기 위하여 다차원 벤치마크를 수행한다. 벤치마크는 인공지능 모델을 구성하는 대표적인 연산자로 구성된 4가지의 모델사용하며, 각 벤치마크는 10단계로 이루어져 있다. 단계마다 연산량과 메모리 접근량이 증가하도록 구성되어 있어, 다양한 크기의 모델에 대응하도록 설계되어 있다. 제안하는 내용을 평가하기 위하여 라즈베리 파이 시리즈를 사용하였다. 실험을 통하여 리소스 셀의 산출의 정확도와 분산처리 성능 그리고 리소스 셀의 배터리 효율을 분석하여 리소스 셀 구조가 엣지 환경에서 분산 인공지능을 운용하는데 유효함을 확인한다.

□ 핵심주제어

엣지 컴퓨팅, 분산 인공지능, 사물인터넷, 벤치마크, 자원제약성, 이기종 환경